A 小红打舞萌

比较规则是：

1. 数字越大越难。
2. 数字相同时，带 + 比不带 + 难。

转化一下，$score(x) = Digit \times 2 + [x.back() ==' +']$。

```
void solve() {
    string a, b;
    cin >> a >> b;
    int A = 0, B = 0;
    for (auto v : a) {
        if (isdigit(v)) A = A * 20 + 2 * (v - '0');
        else A++;
    }
    for (auto v : b) {
        if (isdigit(v)) B = B * 20 + 2 * (v - '0');
        else B++;
    }
    cout << (A > B ? "Yes" : "No") << "\n";
}
```

Fence 1

B 小红写谱

我们不难发现两个规律：

- 连续的同一个键没有贡献。
- 顺时针/逆时针走过每一个键的贡献是最小的。

所以只需要统计哪些键位出现过即可，然后以每一个出现的键为起点，旋转一圈的贡献。

当然我们不难发现，这个最优贡献就相当于整一圈减去相差最远的两个键之间的距离。

```
void solve() {
    int n;
    cin >> n;
    vector<bool> vis(9);
    for (int i = 0, x; i < n; i++) {
        cin >> x;
        vis[x] = 1;
    }

    vector<int> a;
    for (int i = 0; i < 9; ++i) if (vis[i]) a.pb(i);

    int mx = 0, p = a.back() - 8;

    for (int x : a) {
        mx = max(mx, x - p);
        p = x;
    }

    cout << 8 - mx << "\n";
}
```

C 小红出勤

对于任意一首必选歌 x ，它的游玩次数 $cnt[x]$ 必须至少等于出勤次数 $s \to s \leq min(cnt[\text{必玩}])$

每次出勤最多玩 p 首歌，也就是说 s 次出勤能够提供的总游玩容量为 $s \times p$ 。所有的游玩记录总和 tot 必须能被塞进这 s 次出勤中 $\to s \geq \lceil \frac{tot}{p} \rceil$

```cpp
void solve() {
    int n, p, k;
    cin >> n >> p >> k;

    map<string, int> cnt;
    ll tot = 0;

    for (int i = 0; i < n; i++) {
        string s;
        int a;
        cin >> s >> a;
        cnt[s] = a;
        tot += a;
    }

    int mx = inf;
    for (int i = 0; i < k; i++) {
        string s;
        cin >> s;
        mx = min(mx, cnt[s]);
    }

    if (p < k) {
        cout << -1 << "\n";
        return;
    }

    int mn = (tot + p - 1) / p;

    if (mn > mx) cout << -1 << "\n";
    else cout << mn << " " << mx << "\n";
}
```

D 小红做梦

主播觉得最阴的一题，给我搞完气爆两颗。

不难想到去枚举除的次数，然后 $+c$ 的次数即为满足 $cur + c * t \in [1005..., 1010...]$ 。

我们只需要用 l 和 r 模拟两个边界即可。

```
void solve() {
    ll x, a, b, c;
    cin >> x >> a >> b >> c;

    ll ans = INF, cur = x;

    auto ssolve = [&](ll X) {
        ll l = 1005, r = 1010;
        while (1) {
            if (X <= l) {
                ll n = (l - X + c - 1) / c;
                if (n * c + X <= r) return n;
            }
            l *= 10, r *= 10;
        }
    };

    for (int i = 0; i <= 17; ++i) {
        ans = min(ans, a * i + ssolve(cur) * b);
        cur /= 10;
    }

    cout << ans << "\n";
}
```

E 小红越级（easy）

法1（$O(n)$）：

为了算不舒适，我们只需要算舒适的个数就行了。

对于一首歌，它能提供的舒适范围为 $[a-1,a+1]\cup[b-1,b+1]$ 。

那么对于多个区间 $+1$，然后查单点，很容易想到使用差分数组。

对于每个有效区间 $[L,R]$ ，执行 $diff[L]++$ 和 $diff[R+1]--$ 。

处理完所有歌曲后，对 diff 数组求前缀和，$pre[k]$ 即为能力为 k 时的舒适歌曲数量。

```
void solve() {
    int n, q;
    cin >> n >> q;

    vector<int> diff(n + 2, 0);

    for (int i = 0; i < n; ++i) {
        int a, b;
        cin >> a >> b;

        int l1 = a - 1, r1 = a + 1, l2 = b - 1, r2 = b + 1;

        if (r1 + 1 >= l2) {
            int L = max(1, l1), R = min(n, r2);
            if (L <= R) {
                diff[L]++;
                if (R + 1 <= n + 1) diff[R + 1]--;
            }
        } else {
            int L1 = max(1, l1), R1 = min(n, r1);
            if (L1 <= R1) {
                diff[L1]++;
                if (R1 + 1 <= n + 1) diff[R1 + 1]--;
            }

            int L2 = max(1, l2), R2 = min(n, r2);
            if (L2 <= R2) {
                diff[L2]++;
                if (R2 + 1 <= n + 1) diff[R2 + 1]--;
            }
        }
    }

    vector<int> pre(n + 1);
    int cur = 0;
    for (int i = 1; i <= n; ++i) {
        cur += diff[i];
        pre[i] = cur;
    }

    for (int i = 0; i < q; ++i) {
        int k;
```

```
        cin >> k;
        cout << n - pre[k] << " ";
    }
    cout << "\n";
}
```

Fence 5

法2（$O(qlogn)$）：

对于每次询问 k，我们需要计算有多少个区间覆盖了点 k。

覆盖点 k 的区间数量 = (起点 $\leq k$ 的区间数量) - (终点 $< k$ 的区间数量)。

```
#include<bits/extc++.h>
using namespace __gnu_pbds;
using Tree = tree<PII, null_type, less<PII>, rb_tree_tag, tree_order_statistics_node_update>;

void solve() {
    int n, q;
    cin >> n >> q;

    Tree L, R;
    int id = 0;

    auto insert = [&](int l, int r) {
        L.insert({l, ++id});
        R.insert({r, ++id});
    };

    for (int i = 0; i < n; ++i) {
        int a, b;
        cin >> a >> b;

        int l1 = a - 1, r1 = a + 1;
        int l2 = b - 1, r2 = b + 1;

        if (r1 + 1 >= l2) {
            insert(min(l1, l2), max(r1, r2));
        } else {
            insert(l1, r1);
            insert(l2, r2);
        }
    }

    for (int i = 0; i < q; ++i) {
        int k;
        cin >> k;
        cout << n - L.order_of_key({k, inf}) + R.order_of_key({k, -inf}) << " ";
    }
    cout << "\n";
}
```

Fence 6

F 小红开机厅

由三角不等式可知，平面上任意一点到 A, B 的距离之和 $K \geq L$ ，且 K 与 L 同奇偶。

- 情况 1 （K=L）：
  - 满足条件的点构成了以 A, B 为对角顶点的实心矩形区域。
  - 该矩形内的整数点数量为 $(|xa - xb| + 1) \times (|ya - yb| + 1)$ 。
- 情况 2 （$K > L$）：
  - 满足条件的点构成了围绕该矩形的一个环状边界。
  - 该边界上的整数点数量为 $2K$ 。（读者自证不难）

哈希表记录机厅距离，排一下即可。

```cpp
void solve() {
    int n;
    cin >> n;

    ll xa, ya, xb, yb;
    cin >> xa >> ya >> xb >> yb;

    ll X = llabs(xa - xb), Y = llabs(ya - yb);
    ll L = X + Y;

    int ch = 0;
    if (xa == xb && ya == yb) {
        ch = 1;
    } else {
        ch = 2;
    }

    vector<ll> qs(n);
    map<ll, int> cnt;

    for (int i = 0; i < n; ++i) {
        ll x, y;
        cin >> x >> y;
        ll dist = llabs(x - xa) + llabs(y - ya) + llabs(x - xb) + llabs(y - yb);
        qs[i] = dist;
        cnt[dist]++;
    }

    for (int i = 0; i < n; ++i) {
        ll K = qs[i];
        ll tot = 0;

        if (K == L) tot = (X + 1) * (Y + 1);
        else tot = 2 * K;

        ll ex = 0;

        if (K == L) ex += ch;
```

```

        if (cnt.count(K)) ex += cnt[K];

        cout << (tot - ex) << " ";
    }
    cout << "\n";
}
```

G 小红越级（hard）

难度为 p 能力为 k，不舒适度为 $g(p,k)=max(0,|p-k|-1)$

对于每首歌，它的最低不舒适度为 $f_i(k)=min(g(a_i,k),g(b_i,k))$

我们要计算所有歌的总和 $F=\sum f(i)$

$f_i(k)$ 是分段线性的。我们可以通过维护其二阶差分（斜率的变化率）来快速计算。记 $diff[x]$ 为在 x 处斜率的变化量

对于一首歌 (a,b)，我们考察 k 从小到大变化时 $f_i(k)$ 的斜率变化：

- 两个“舒适区间”重叠或相接（$b-a\le 2$）：

  此时 $[a-1,a+1]$ 和 $[b-1,b+1]$ 的并集是一个连续区间 $[a-1,b+1]$。

  在此区间内代价为 0，左边斜率 -1，右边斜率 1。

  - 在 a 处（对应 $k=a$），斜率从 -1 变为 0（增加 1）。
  - 在 $b+2$ 处（对应 $k=b+2$），斜率从 0 变为 1（增加 1）。
- 两个区间分离（$b-a>2$）

  此时有两个零代价区间 $[a-1,a+1]$ 和 $[b-1,b+1]$，中间有个峰。

  在 a 左侧选 a，在 b 右侧选 b。在中间某个位置 m，策略从选 a 切换到选 b。

  - 在 a 处：斜率 -1 $\to 0$ （增加 1）。
  - 在 $a+2$ 处：斜率 $0\to 1$ （增加 1）。
  - 切换点：位于中点 $m=(a+b)/2$ 附近。
    - 若 $a+b$ 是偶数，峰值正好在 m。$k=m$ 时选 a 或 b 代价一样。$k=m$ 属于左坡（斜率 1），$k=m+1$ 属于右坡（斜率 -1）。所以在 $m+1$

      处斜率减少 2。
    - 若 $a+b$ 是奇数，中间有一段平顶 $[m,m+1]$。 $m+1$ 处斜率 $1\to 0$ （减少 1），$m+2$ 处斜率 $0\to$ -1 （减少 1）。
  - 在 b 处：斜率 -1 $\to 0$ （增加 1）。
  - 在 $b+2$ 处：斜率 $0\to 1$ （增加 1）。

```cpp
void solve() {
    int n, q;
    cin >> n >> q;

    vector<int> diff(n + 10, 0);

    ll cur = 0;

    for (int i = 0; i < n; ++i) {
        int a, b;
        cin >> a >> b;

        cur += (a - 1);

        if (b - a <= 2) {
            diff[a] += 1;
```

```
            if (b + 2 <= n + 5) diff[b + 2] += 1;
        } else {
            diff[a] += 1;
            if (a + 2 <= n + 5) diff[a + 2] += 1;

            int sum = a + b;
            int m = sum / 2;
            if (sum % 2 == 0) {
                if (m + 1 <= n + 5) diff[m + 1] -= 2;
            } else {
                if (m + 1 <= n + 5) diff[m + 1] -= 1;
                if (m + 2 <= n + 5) diff[m + 2] -= 1;
            }

            if (b <= n + 5) diff[b] += 1;
            if (b + 2 <= n + 5) diff[b + 2] += 1;
        }
    }

    vector<ll> ans(n + 1);

    ll s = -n;

    for (int k = 1; k <= n; ++k) {
        s += diff[k];
        cur += s;
        ans[k] = cur;
    }

    for (int i = 0; i < q; ++i) {
        int k;
        cin >> k;
        cout << ans[k] << " ";
    }
    cout << "\n";
}
```

头文件

```cpp
//Another
#include<bits/stdc++.h>
#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define all(a) a.begin(), a.end()
#define rall(a) a.rbegin(), a.rend()
using namespace std;

using ll = long long;
using ld = long double;
using ull = unsigned long long;
using i128 = __int128;
using PII = pair<int, int>;
using PLL = pair<ll, ll>;
using TIII = tuple<int, int, int>;
using TLLL = tuple<ll, ll, ll>;

constexpr int inf = (ll)1e9 + 7;
constexpr ll INF = (ll)2e18 + 9;
// constexpr ll INF = (ll)4e18;
// constexpr ll MOD = 1e9 + 7;
constexpr ll MOD = 998244353;
constexpr ld eps = 1e-10;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ull randint(ull l, ull r) {uniform_int_distribution<unsigned long long>
dist(l, r); return dist(rng);}

void init() {

}

void solve() {

}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    init();

    int t = 1;
    cin >> t;
    for (int _ = 1; _ <= t; ++_) {
        solve();
    }

    return 0;
```

```
}
```

Fence 9